

STACK DE MODELISATION UML

Conception & Diagrammes

Projet : Site Web Association Tifaouine

Outil principal	PlantUML 1.2025.10
Diagrammes	Use Case Classes Sequence
Stack technique	React + Node.js (Express) + PostgreSQL
Version document	1.0 — Mars 2026
Auteur	Equipe conception — Association Tifaouine

Document de reference pour la phase de conception UML du projet.

1. Contexte et Objectifs de la Modelisation

Ce document constitue le cahier de la stack de modelisation UML realise dans le cadre de la conception du site web de l'Association Tifaouine. Il regroupe l'ensemble des diagrammes produits, les conventions adoptees et les decisions de conception justifiees.

L'objectif de cette phase de modelisation est d'etablir une vision claire et partagee de l'architecture fonctionnelle et structurelle du systeme avant de demarrer le developpement.

1.1 Perimetre du systeme modelise

- Site web institutionnel bilingue (Arabe / Francais) de l'association
- Gestion des projets, evenements, dons, benevoles et partenariats
- Interface d'administration back-office (CRUD complet)
- API REST Node.js + Express consommee par le frontend React
- Base de donnees PostgreSQL avec migrations Sequelize

1.2 Diagrammes produits

#	Diagramme	Outil	Statut
D 1	Use Case (cas d'utilisation)	PlantUML	Finalise
D 2	Diagramme de Classes	PlantUML	Finalise
D 3	Diagramme de Sequence — Don	PlantUML	Finalise
D 4	Diagramme de Deploiement	PlantUML	A produire
D 5	Diagrammes d'Activite	PlantUML	A produire

2. Conventions de Modelisation

2.1 Outil et environnement

Outil UML	PlantUML 1.2025.10
Editeur	VS Code + extension PlantUML (jebbs)
Rendu local	Java JRE 8+ + Graphviz
Rendu en ligne	http://www.plantuml.com/plantuml/uml
Format sortie	.puml (source) + .png (export)
Versionnage	Git — dossier /conception/ a la racine du repo

2.2 Regles de nommage

Element	Convention	Exemple
Fichier .puml	type_sujet.puml	seq_effectuer_donation.puml
Acteurs	PascalCase	Visiteur, Administrateur
Classes	PascalCase	DonFinancier, Utilisateur
Attributs	camelCase avec visibilite	- passwordHash : String
Methodes	camelCase()	+ validerDemande() : void
Alias Use Case	UCxx	UC01, UC06b2
Participants seq.	PascalCase court	API, BD, UI

2.3 Notation synchrone / asynchrone (Sequence)

Notation	Type	Usage dans le projet
A -> B	Synchrone (fleche pleine)	Appels HTTP REST, requetes SQL, paiement
A ->> B	Asynchrone (fleche ouverte)	Notifications email/push, alertes Admin
A --> B	Retour (tirets)	Reponses HTTP, resultats SQL
A -->> B	Retour asynchrone	Callback eventuel apres notification

3. Diagramme de Cas d'Utilisation (Use Case)

3.1 Description

Le diagramme de cas d'utilisation modelise les interactions entre les acteurs et les fonctionnalites du systeme. Il constitue le point d'entree de la conception et sert de reference pour identifier les besoins fonctionnels.

3.2 Acteurs identifies

Acteur	Type	Role principal
Visiteur	Acteur primaire interne	Navigation publique, soumission formulaires, donations
Benevole	Acteur primaire (herite V)	Herite de Visiteur — membre actif benevole
Membre	Acteur primaire (herite V)	Herite de Visiteur — membre de l'association
Administrateur	Acteur secondaire interne	Gestion complete du back-office (CRUD)
Sys. Paiement	Acteur externe	Traitement des transactions financieres (PayPal/Stripe)

3.3 Cas d'utilisation principaux

ID	Use Case	Zone	Acteur
UC-01	Consulter le site	Publique	Visiteur
UC-02	Demander a etre benevole	Publique	Visiteur
UC-03	Contacter l'association	Publique	Visiteur
UC-04	Devenir membre	Publique	Visiteur
UC-05	Demander a etre partenaire	Publique	Visiteur
UC-06	Effectuer une donation	Don	Visiteur
UC-06a	Don vers tresorerie generale	Don	Visiteur
UC-06b	Don sur projet specifique	Don	Visiteur
UC-07	Gerer le contenu (CMS)	Administration	Admin

ID	Use Case	Zone	Acteur
UC-08	Gerer membres et benevoles	Administration	Admin
UC-09	Traiter demandes partenariat	Administration	Admin
UC-10	Initier un partenariat	Administration	Admin
UC-11	Valider don materiel	Administration	Admin
UC-12	Gerer les roles	Administration	Admin

3.4 3.5 Diagramme Use Case — version finale

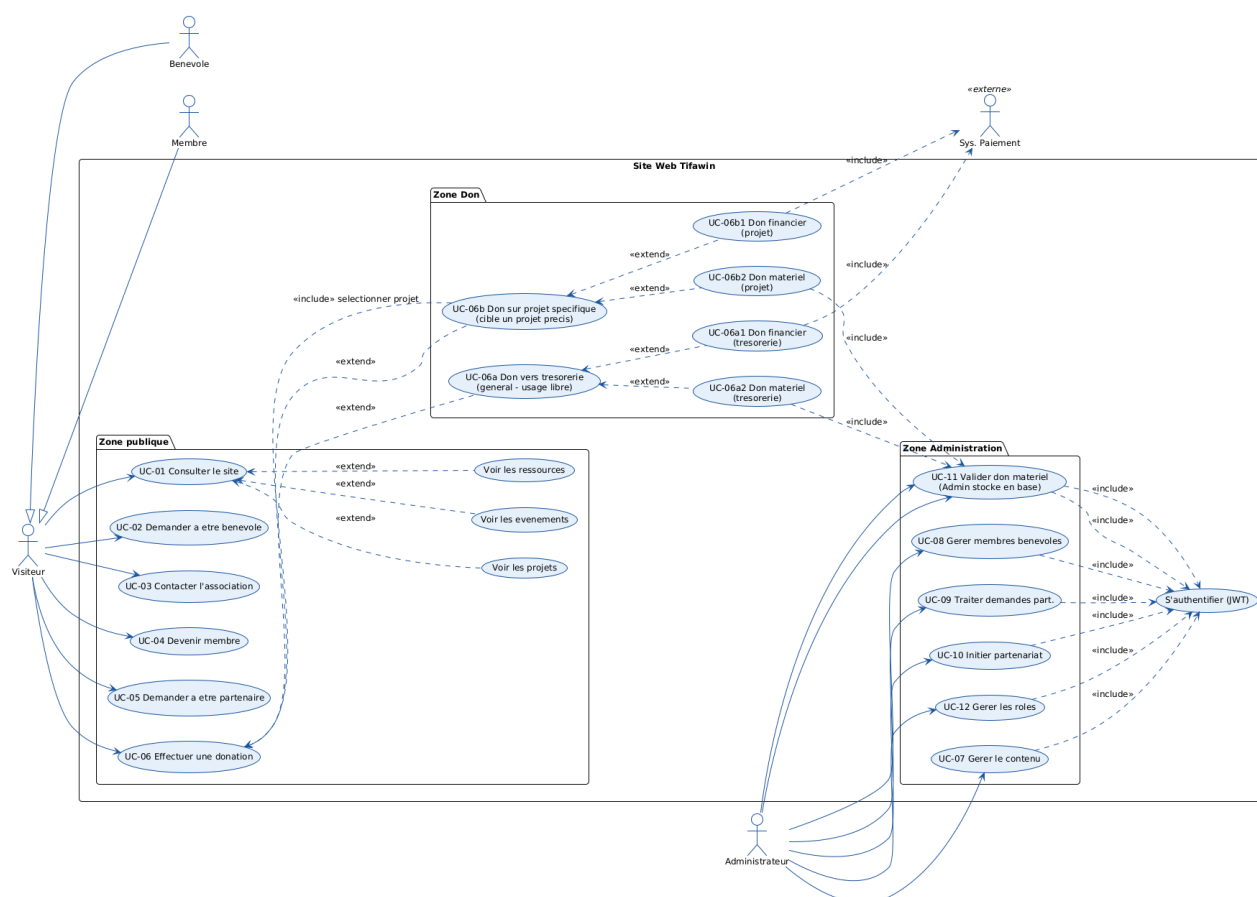


Figure 1 — Diagramme de Cas d'Utilisation — Association Tifaouine v2

4. Diagramme de Classes

4.1 Description

Le diagramme de classes modelise la structure statique du systeme : les entites metier, leurs attributs, leurs methodes et les relations qui les lient. Il sert de base directe au schema de base de donnees PostgreSQL et aux modeles Sequelize.

4.2 Organisation en couches

Couche	Classes	Couleur
Couche 1 — Utilisateurs	Utilisateur (abstract), Membre, Benevole, Admin	Bleu fonce
Couche 2 — Contenu Metier	Domaine, Projet, Evenement, Media, Ressource	Teal
Couche 3 — Dons	Don (abstract), DonFinancier, DonMateriel	Ambre
Couche 4 — Support	Partenariat, MessageContact, Stat	Corail

4.3 Classes et attributs cles

Utilisateur (abstract)

Attribut / Methode	Type	Description
- id	Integer	Identifiant unique
- nom	String	Nom complet
- email	String	Email (unique, identifiant de connexion)

Admin

Attribut / Methode	Type	Description
- roles	List<String>	Liste des roles (super_admin, editeur...)
-passwordHash	String	Mot de passe hache (bcrypt, salt >= 12)
+ publier(entite)	void	Publication d'un contenu
+ modifier(entite)	void	Modification d'une entite
+ supprimer(entite)	void	Suppression d'une entite
+ gererRoles()	void	Gestion des roles multiples
+ validerDemande()	void	Validation d'une demande (don materiel, benevole...)
+seconnect()	JWT	validation du password hashe dans database

Don (abstract)

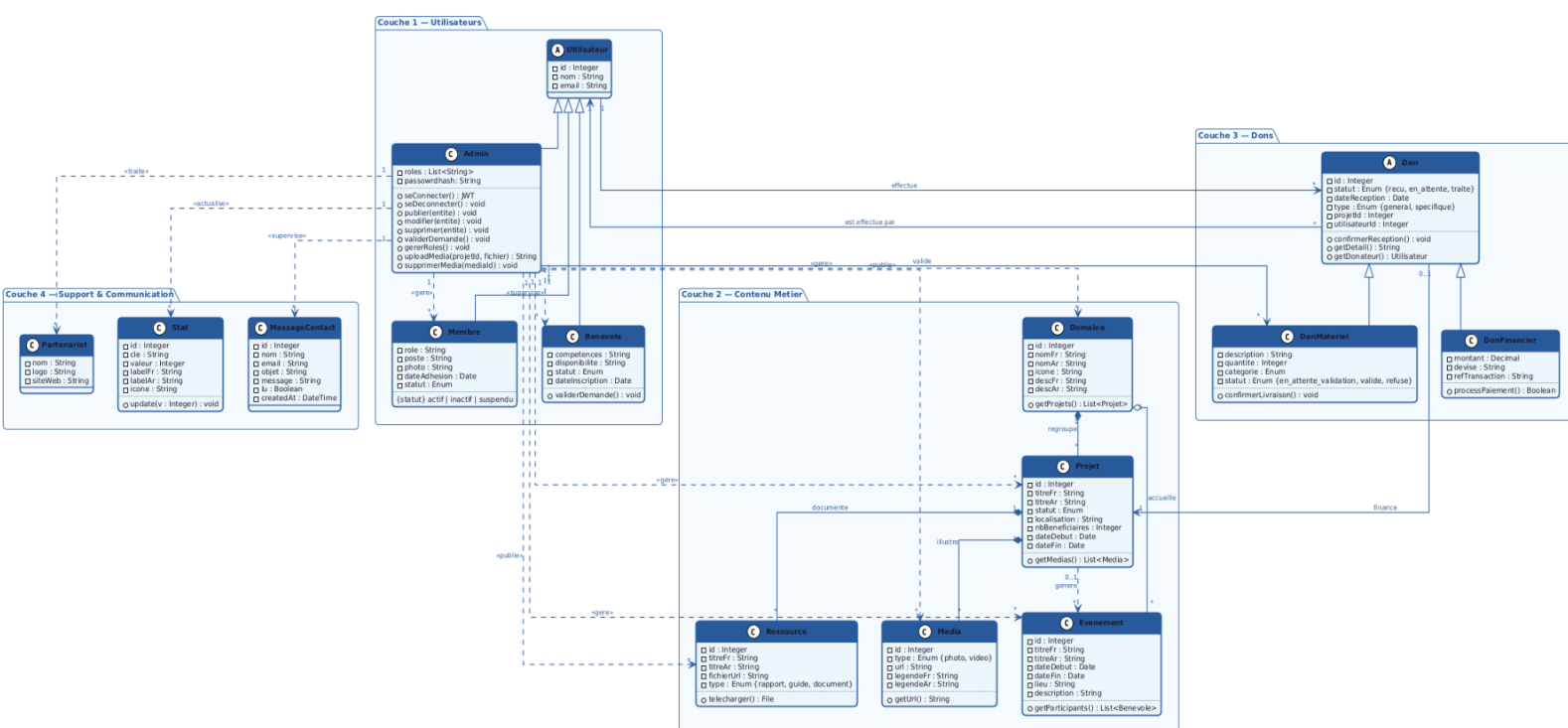
Attribut / Methode	Type	Description
- id	Integer	Identifiant unique
- statut	Enum	recu en_attente traite
- dateReception	Date	Date de reception du don
- type	Enum	general (tresorerie) specifique (projet)
- projetId	Integer	Lien optionnel vers un projet (null si general)
-utilisateurId	Integer	la clé etranger vers la table utilisateur qui a fait le don
+ confirmerReception()	void	Confirmation de reception
+ getDetail()	String	Retourne les details du don

4.4 Relations principales

Relation	Source	Cible	Cardinalite	Verbe
Composition	Domaine	Projet	1 → *	regroupe
Agregation	Domaine	Evenement	1 → *	accueille
Dependance opt.	Projet	Evenement	0..1 → *	genere
Composition	Projet	Media	1 → *	illustre
Agregation	Projet	Ressource	1 → *	documente
Association	Utilisateur	Don	1 → *	effectue
Association	Admin	DonMateriel	1 → *	valide
Association opt.	Don	Projet	0..1 → 1	finance
Association	Admin	Projet+...	1 → *	gere

4.5 Diagramme de Classes — version finale

Figure 2 — Diagramme de Classes — Association Tifaouine v2



5. Diagramme de Sequence — Effectuer une Donation

5.1 Description

Le diagramme de sequence modelise les interactions chronologiques entre les acteurs et les composants du systeme lors de l'execution du cas d'utilisation UC-06 : Effectuer une donation. Ce diagramme couvre les 4 variantes du flux metier.

5.2 Participants

Participant	Couche	Technologie	Role
Utilisateur	Acteur	—	Declencheur du flux de don
Interface	Frontend	React SPA (Vite)	Affichage du formulaire et des confirmations
API Node	Backend	Express.js	Traitement metier, validation, orchestration
Base Donnees	Donnees	PostgreSQL	Persistence des dons, stats, projets

Participant	Couche	Technologie	Role
Sys. Paiement	Externe	PayPal / Stripe	Traitement des transactions financieres
Admin	Acteur	Back-office React	Validation des dons materiels

5.3 Flux couverts par le diagramme

Flux	Description	Type de message cle
Flux 1	Don financier vers tresorerie generale	Sync : processPaiement() bloquant
Flux 2	Don materiel vers tresorerie generale	Async : notification Admin par email/push
Flux 3	Don financier sur projet specifique	Sync : processPaiement() bloquant
Flux 4	Don materiel sur projet specifique	Async : notification Admin par email/push

5.4 Nature des messages — Synchrone vs Asynchrone

Message	Nature	Justification
Utilisateur → Interface	SYNCHRONE —>	L'utilisateur attend l'affichage du formulaire
Interface → API (POST /dons)	SYNCHRONE —>	React attend le 201 Created pour confirmer
API → Base de donnees	SYNCHRONE —>	Requetes SQL bloquantes (INSERT, UPDATE, SELECT)
API → Sys. Paiement	SYNCHRONE —>	L'API doit attendre la confirmation du paiement
API → Admin (notification)	ASYNCHRONE —>>	L'API a deja repondu — email/push en parallele
API → Utilisateur (statut)	ASYNCHRONE —>>	Notification differee apres decision Admin

5.5 Fragments UML utilises

Fragment	Condition	Contenu
alt (destination)	[tresorerie] / [projet specifique]	Choix de la destination avant saisie du formulaire
alt (type)	[Don Financier] / [Don Materiel]	Branchement vers le flux paiement ou le flux validation
opt (validation)	[type = Don Materiel]	Bloc de validation Admin declenche uniquement pour materiel

Fragment	Condition	Contenu
alt (decision)	[Admin valide] / [Admin refuse]	Decision Admin — UPDATE statut en base
opt (stat)	[don valide ou financier traite]	Mise a jour du compteur Stat en base

5.6 Endpoints REST impliqués

GET /api/projets	→ charge la liste des projets actifs
POST /api/dons/financier	→ creer un DonFinancier (statut=recu)
POST /api/dons/materiel (statut=en_attente_validation)	→ creer un DonMateriel
GET /api/dons/materiel/{id}	→ Admin consulte les details d'un don
PATCH /api/dons/materiel/{id}/valider	→ Admin valide — UPDATE statut=valide
PATCH /api/dons/materiel/{id}/refuser	→ Admin refuse — UPDATE statut=refuse
PUT /api/stats/total_dons	→ mise a jour du compteur d'impact